



“ 소프트웨어로 하드웨어를 제어해 보자 ”

IOT 실습심화

4주차 : PWM을 이용한 서보모터 제어하기

4주차 수업은 서보모터

HTTP
Server

HTTP
Client

응용
프로그래밍

스마트온도감시시스템 (project.py)
with HTTP Server

시스템
프로그래밍

Linux OS

소켓

네트워크
TCP/IP

네트워크(소켓)
프로그래밍

LED
(RGB)
소자

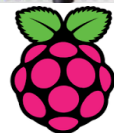
PWM
(서보
모터)

Buzzer
소자

LCD
소자

Sensor
소자

Camera
소자



RaspberryPi

python™

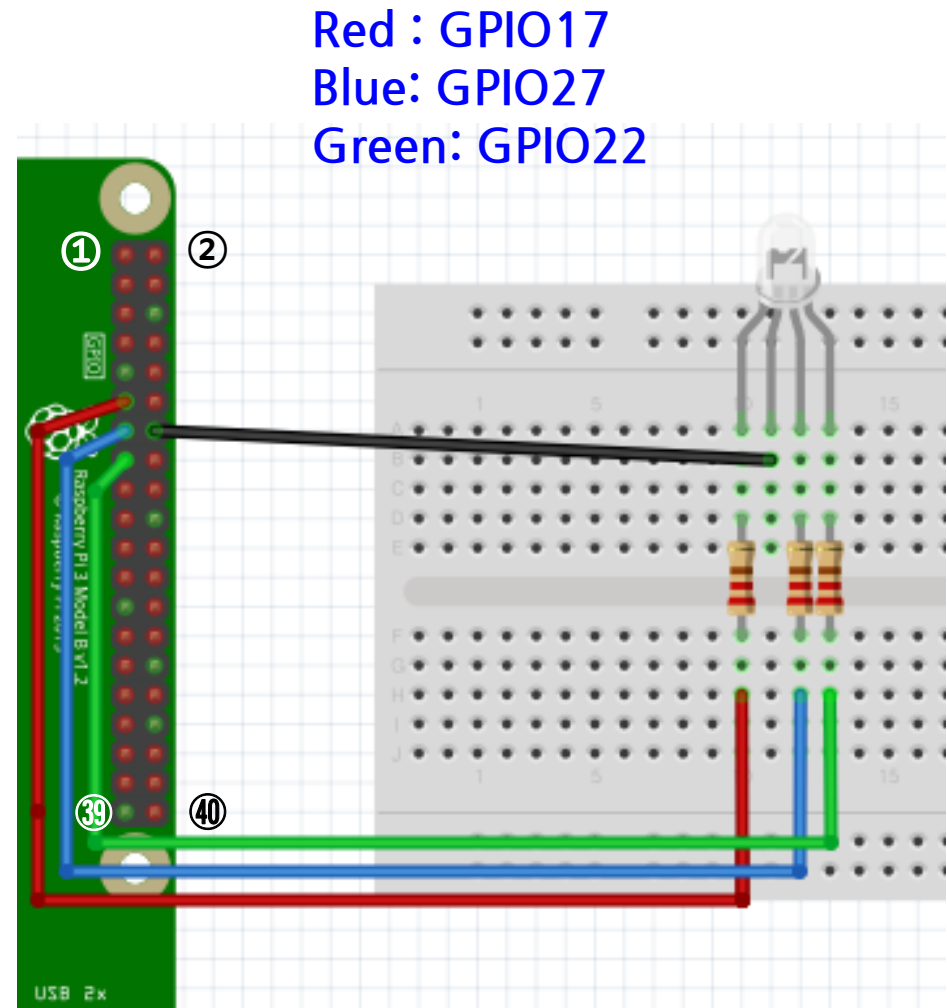
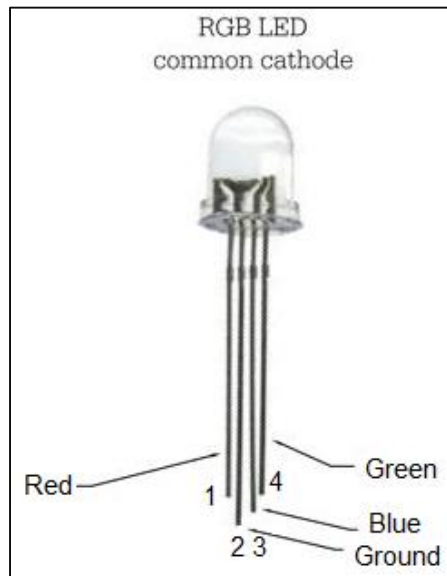
“ 소프트웨어로 하드웨어를
제어해 보자”



3주차 피드백) RGB LED 회로 구성하기

RGB LED 제어하기 회로 구성

라즈베리파이		3색 LED
GPIO#17 (11번)	저항	1 (Red)
GND(14번)		2(GND)
GPIO#27 (13번)	저항	3(Blue)
GPIO#22 (15번)	저항	4(Green)



3주차 피드백) RGB LED 회로 구성하기

□ GPIO pin map 확인방법 (BCM mode)

- BCM(Broadcom) 모드

라즈베리파이 칩(Broadcom SOC)에 직접 할당된
GPIO 핀 번호(실제 핀 번호)를 사용하겠다는 설정
*GPIO 실습에 사용

- GPIO pin map 확인하는 명령어

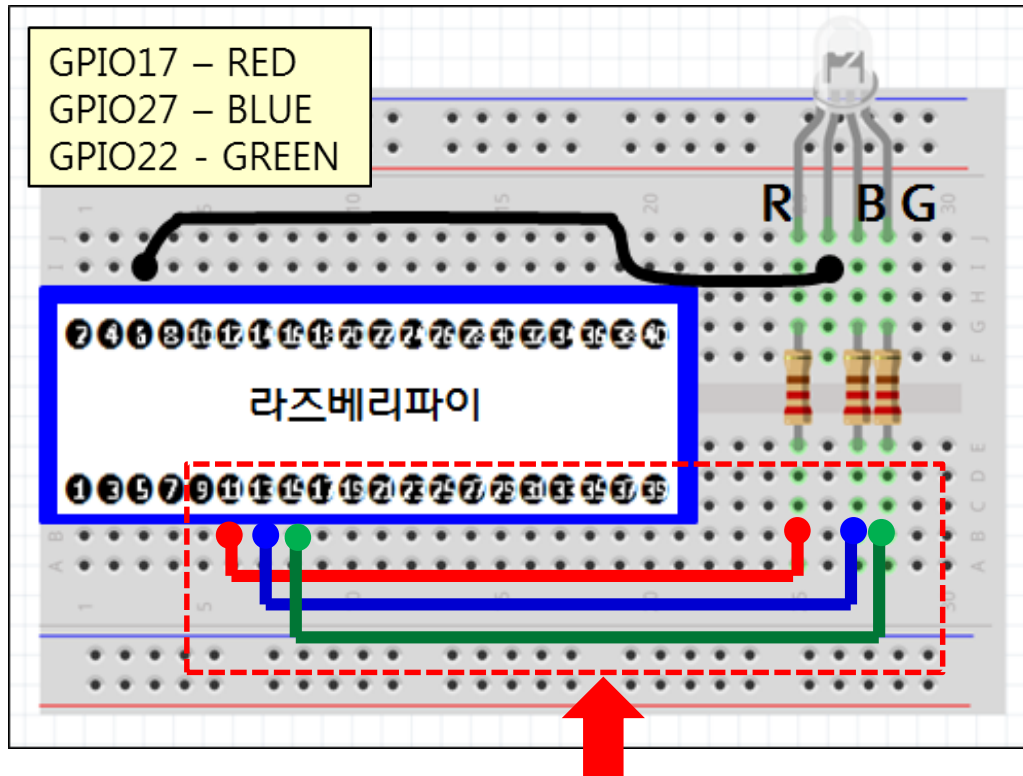
```
pi@raspberrypi:~ $ pinout
```

Red : GPIO17
Blue: GPIO27
Green: GPIO22

BCM		BCM	
3V3	(1)	(2)	5V
GPIO2	(3)	(4)	5V
GPIO3	(5)	(6)	GND
GPIO4	(7)	(8)	GPIO14
GND	(9)	(10)	GPIO15
GPIO17	(11)	(12)	GPIO18
GPIO27	(13)	(14)	GND
GPIO22	(15)	(16)	GPIO23
3V3	(17)	(18)	GPIO24
GPIO10	(19)	(20)	GND
GPIO9	(21)	(22)	GPIO25
GPIO11	(23)	(24)	GPIO8
GND	(25)	(26)	GPIO7
GPIO0	(27)	(28)	GPIO1
GPIO5	(29)	(30)	GND
GPIO6	(31)	(32)	GPIO12
GPIO13	(33)	(34)	GND
GPIO19	(35)	(36)	GPIO16
GPIO26	(37)	(38)	GPIO20
GND	(39)	(40)	GPIO21

3주차 피드백) RGB LED 회로 구성하기

- 중간고사 출제 : RGB 동작회로 구성하고, RGB LED ON/OFF 제어 코딩하기
 - * GPIO port는 변경될 수 있음



3V3	(1)	(2)	5V
GPIO2	(3)	(4)	5V
GPIO3	(5)	(6)	GND
GPIO4	(7)	(8)	GPIO14
GND	(9)	(10)	GPIO15
GPIO17	(11)	(12)	GPIO18
GPIO27	(13)	(14)	GND
GPIO22	(15)	(16)	GPIO23
3V3	(17)	(18)	GPIO24
GPIO10	(19)	(20)	GND
GPIO9	(21)	(22)	GPIO25
GPIO11	(23)	(24)	GPIO8
GND	(25)	(26)	GPIO7
GPIO0	(27)	(28)	GPIO1
GPIO5	(29)	(30)	GND
GPIO6	(31)	(32)	GPIO12
GPIO13	(33)	(34)	GND
GPIO19	(35)	(36)	GPIO16
GPIO26	(37)	(38)	GPIO20
GND	(39)	(40)	GPIO21

3주차 피드백) RGB ON/OFF 제어 코드

- ❑ rgb.py (1초 간격으로 RED → GREEN → BLUE 점등되기)
(※ RGB 제어코드는 중간고사에 출제가 되니 확실히 이해하기!!)

```
# RGBLED 기본제어 방법:blink동작
from gpiozero import RGBLED
import time

# RGBLED pin 지정후 객체 생성
rgb = RGBLED(red=17, green=22, blue=27)

print("RGB LED 순서대로 blink하기")
try:
    while True:
        print("RED ON")
        rgb.color = (1, 0, 0) # (r, g, b)
        time.sleep(1)
        print("GREEN ON")
        rgb.color = (0, 1, 0)
        time.sleep(1)
        print("BLUE ON")
        rgb.color = (0, 0, 1)
        time.sleep(1)
except KeyboardInterrupt:
    pass
finally:
    # 예외처리 종료시 모든 리소스 cleanup 하기
    rgb.close()
    print("Program End")
```

1. gpiozero 라이브러리의 RGBLED 클래스이용

- RGBLED 모듈을 쉽게 제어할 수 있도록 만든 전용 도구
- 핵심 제어 수단: color 속성(Property)
- 기본적으로 softPWM으로 동작
전류의 양을 미세 조정하여 밝기조절

2. rgb.color = (값) 형식,

* color는 속성(property)

함수로 호출하는 것이 아니라, 일반 변수처럼 값을 직접 읽거나 바꿀 수 있도록 설계

- 데이터 형태: 3개의 소수점을 가진 튜플 형태 (Red, Green, Blue)
- 값의 범위: 각 색상 채널의 범위는 0.0(완전히 꺼짐)부터 1.0(최대 밝기)까지

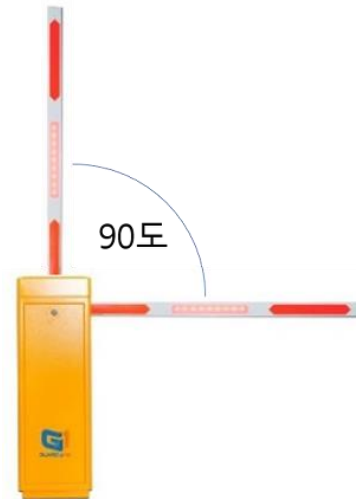
3주차 피드백) PWM으로 LED 밝기 제어하기

PWM(Pulse Width Modulation: 펄스 폭 변조)은 디지털 신호를 이용해 아날로그 회로처럼 제어하는 방법입니다.

펄스의 폭을 조정하여 LED의 밝기와 서보모터의 방향 제어에 사용됩니다



<https://www.ilovepc.co.kr/news/>



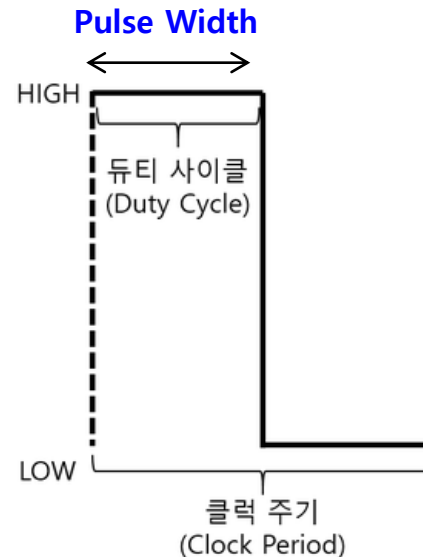
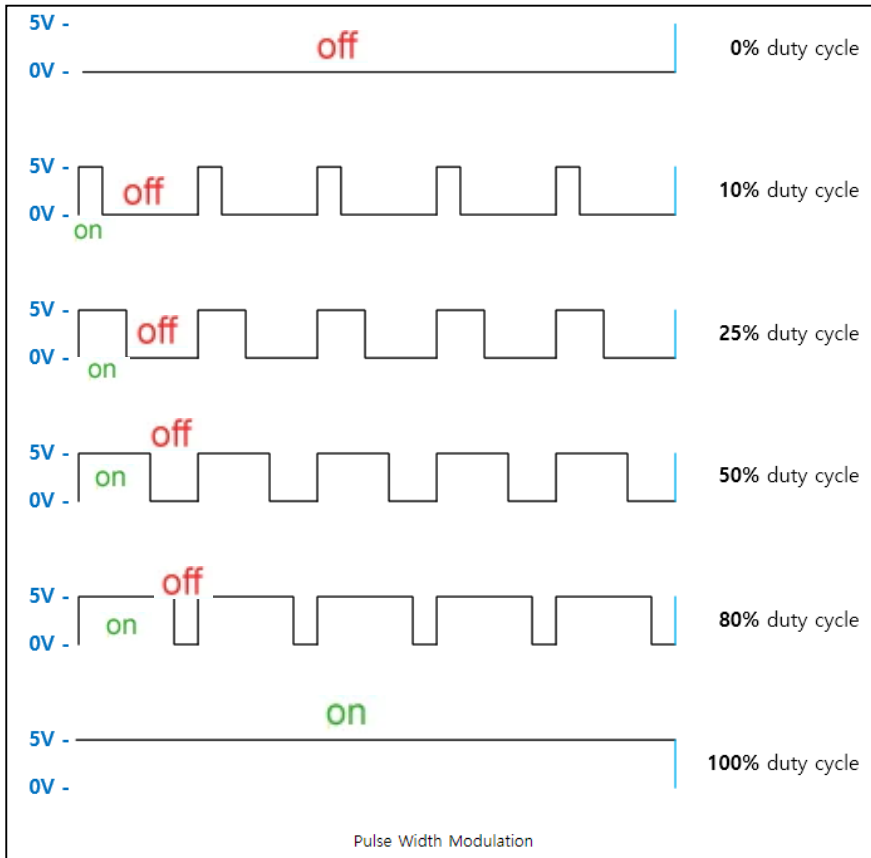
<https://t1.daumcdn.net/cfile/tistory>

3주차 피드백) PWM으로 LED 밝기 제어하기

“좀 더 밝게 ... 좀 더 어둡게...” 디지털로 아날로그를 제어해 보자
라즈베리파이와 함께 PWM을 사용하여 아날로그 출력을 다룹니다

□ 소프트웨어 PWM을 이용한 LED 밝기 제어

PWM(Pulse Width Modulation): 펄스 폭 변조로 디지털 출력으로 아날로그 회로를 제어하는 기법 → PWM 기능 이용 LED 밝기조절



$$\text{Duty cycle(듀티비)} = \frac{\text{Pulse Width}}{\text{주기(Period)}} \times 100$$

*주기(Period) = 하나의 사이클이 완성되는 시간

$$\text{주파수}(f) = \frac{1}{\text{주기}(t)}$$

*주파수(frequency): 1초동안 반복하는 횟수 (Hz)

3주차 피드백) PWM을 이용한 RGB LED 밝기제어

□ 실습3: 듀티비를 입력 받아 PWM으로 RED LED 밝기 제어하기

* 파일명: red_pwm.py

```
# RGB LED PWM으로 밝기제어하기
from gpiozero import RGBLED

# RGBLED 핀 지정후 객체 생성
rgb = RGBLED(red=17, green=22, blue=27)

print("RGBLED PWM을 이용해 밝기제어하기")
print("="*30)
print("종료하려면 'q' 입력하세요")
print("="*30)
```

```
>>> %Run red_pwm.py
```

```
RGBLED PWM을 이용해 밝기제어하기
=====
종료하려면 'q' 입력하세요
=====
RED 듀티비 입력(0~100):10
RED LED 듀티값 변경:0.1
RED 듀티비 입력(0~100):50
RED LED 듀티값 변경:0.5
RED 듀티비 입력(0~100):90
RED LED 듀티값 변경:0.9
RED 듀티비 입력(0~100):|
```

```
try:
    while True:
        r_input= input("RED 듀티비 입력(0~100):")

        if r_input == 'q': # 종료
            break
        if not(0 <= int(r_input) <= 100): # 예외처리
            print("0~100사이의 정수를 입력하세요")
            continue

        # 입력된 값을 PWM 듀티값(0.0~1.0)으로 변환
        r_pwm = int(r_input)/100.0
        rgb.color = (r_pwm, 0.0, 0.0) # RED LED PWM 듀티값 변경

        print(f"RED LED 듀티값 변경:{r_pwm}")
except KeyboardInterrupt:
    print("사용자 강제 중지로 종료")
finally:
    rgb.close() # 예외처리로 종료시 리소스 반환
    print("프로그램을 종료합니다")
```

3주차 피드백) PWM을 이용한 RGB LED 밝기제어

□ 실습4: RED LED PWM을 이용해 페이딩(Fading) 하기 (또는 Dimming)

* 페이딩 (Fading): 시간이 흐르며 밝기가 '변화하는 과정', Fade-in / Fade-out

* 파일명: red_fading.py

- 조건: RED LED의 밝기 조절하기
GREEN, BLUE는 OFF 상태 유지
- 1) RED LED PWM 듀티비 변화
(0 → 100까지 1씩 증가)
time 간격은 0.02초
 - 2) RED LED PWM 듀티비 변화
(100 → 0까지 -1씩 감소)
time 간격은 0.02초
 - 3) 반복 실행

```
# RGBLED PWM으로 Fading(또는 Dimming) 하기
from gpiozero import RGBLED
import time
```

```
# RGBLED pin 지정 후 객체생성
rgb = RGBLED(red=17, green=22, blue=27)
```

```
print("RGB LED PWM")
print("종료하려면 'q' 입력하세요")

rgb.pulse(fade_in_time=2, fade_out_time=2, on_color=(1,0,0))
# pulse()는 백그라운드 스레드로 동작
```

```
try:
    while True:
        print("RED LED Fade-in")

        for r in range(0, 101):
            # 정수를 PWM 듀티값으로 변환(r/100.0)
            rgb.color = (r/100.0, 0.0, 0.0) # RED 점점 밝게
            time.sleep(0.02)                # Fading 속도 조절

        print("RED LED Fade-out")

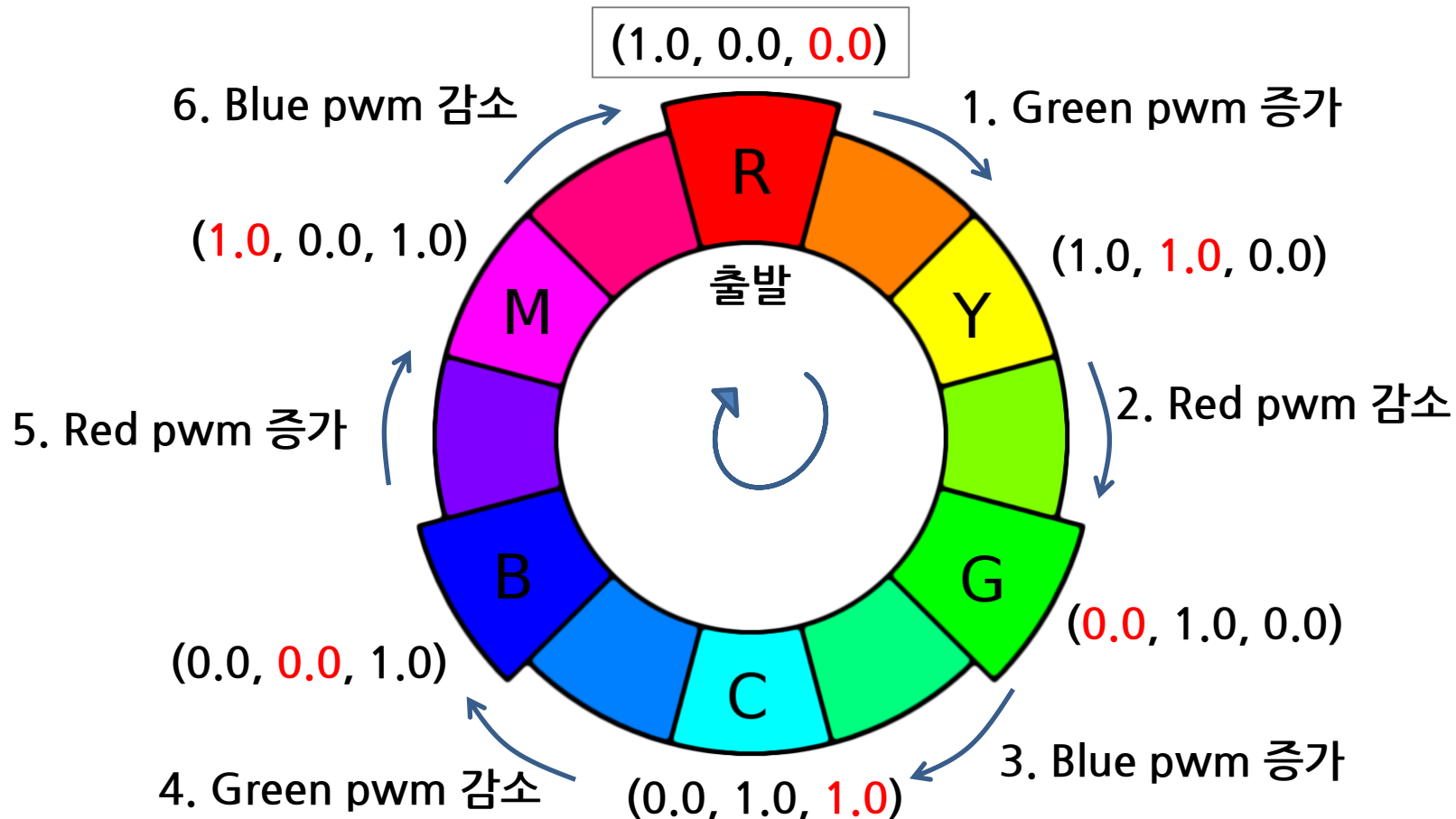
        for r in range(100, -1, -1):
            # 정수를 PWM 듀티값으로 변환(r/100.0)
            rgb.color = (r/100.0, 0.0, 0.0) # RED 점점 어둡게
            time.sleep(0.02)                # Fading 속도 조절

except KeyboardInterrupt: # CTRL-C 종료
    pass
finally:
    rgb.close() # 리소스 반환
    print("프로그램을 종료합니다")
```

3주차 피드백) PWM을 이용한 RGB LED 밝기제어 응용실습

응용실습) PWM 을 사용하여 Color Fading (무지개 색상 그라데이션) 하기

[작동 방식] Red → Yellow → Green → Cyan → Blue → Pink → Red로 RGB fading 제어하기



3주차 피드백) PWM을 이용한 RGB LED 밝기제어 응용실습

응용실습) PWM 을 사용하여 Color Fading (무지개 색상 그라데이션) 하기

* 파일명: 이름_3주차_실습과제.py

3주차 응용실습과제: 무지개 그라데이션

```
from gpiozero import RGBLED
import time
```

RGBLED pin 설정 후 객체 생성

```
rgb = RGBLED(red=17, green=22, blue=27)
```

색상 변환속도 제어

```
DELAY = 0.02
```

```
print("PWM을 이용한 무지개 그라데이션 시작.(종료는 'CTRL+C')")
```

```
try:
```

```
    while True:
```

```
        print("Red->Yellow (GREEN duty 증가)")
```

```
        # (1.0, 0.0, 0.0) -> (1.0, 1.0, 0.0)
```

```
        for g in range(0,101):
```

```
            rgb.color=(1.0, g/100.0, 0.0)
```

```
            time.sleep(DELAY)
```

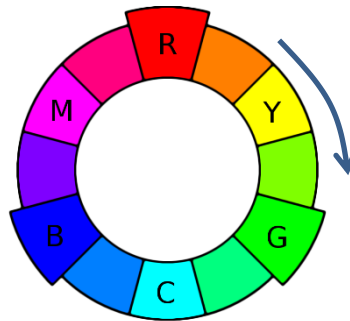
```
        print("Yellow-> Green (RED duty 감소)")
```

```
        # (1.0, 1.0, 0.0) -> (0.0, 1.0, 0.0)
```

```
        for r in range(100, -1, -1):
```

```
            rgb.color = (r/100.0, 1.0, 0.0)
```

```
            time.sleep(DELAY)
```



```
print("Green->Cyan (BLUE duty 증가)")
# (0.0, 1.0, 0.0) -> (0.0, 1.0, 1.0)
for b in range(0,101):
    rgb.color= (0.0, 1.0, b/100.0)
    time.sleep(DELAY)
```

```
print("Cyan->Blue (GREEN duty 감소)")
# (0.0, 1.0, 1.0) -> (0.0, 0.0, 1.0)
for g in range(100, -1, -1):
    rgb.color= (0.0, g/100.0, 1.0)
    time.sleep(DELAY)
```

```
print("Blue->Pink (RED duty 증가)")
# (0.0, 0.0, 1.0) -> (1.0, 0.0, 1.0)
for r in range(0,101):
    rgb.color= (r/100.0, 0.0, 1.0)
    time.sleep(DELAY)
```

```
print("Pink->Red (BLUE duty 감소)")
# (1.0, 0.0, 1.0) -> (1.0, 0.0, 0.0)
for b in range(100, -1, -1):
    rgb.color= (1.0, 0.0, b/100.0)
    time.sleep(DELAY)
```

```
except KeyboardInterrupt:
```

```
    pass
```

```
finally:
```

```
    rgb.close()
```

```
    print("프로그램을 종료합니다")
```

4주차. PWM을 이용한 서보모터 제어

- 4주차에는 파이썬으로 서보모터를 제어해 봅시다!



IoT 수업: 서보모터의 다양한 응용 분야

Smart Home (스마트 홈)

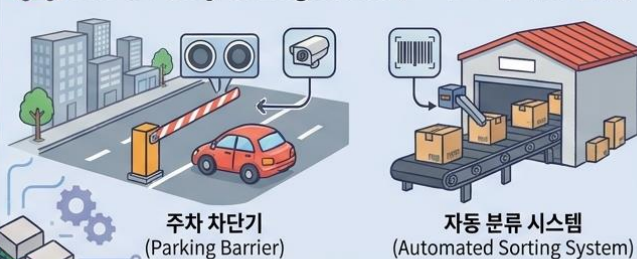


스마트 도어락
(Smart Door Lock)

자동 커튼/블라인드
(Automatic Curtains)

자동 커튼/블라인드

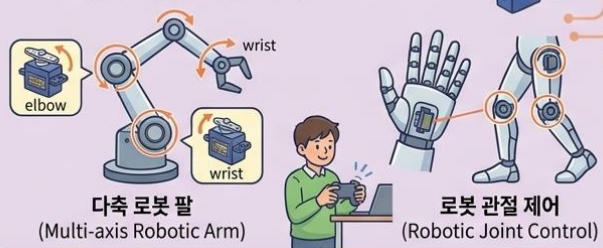
Smart City & Logistics (스마트 시티 & 물류)



주차 차단기
(Parking Barrier)

자동 분류 시스템
(Automated Sorting System)

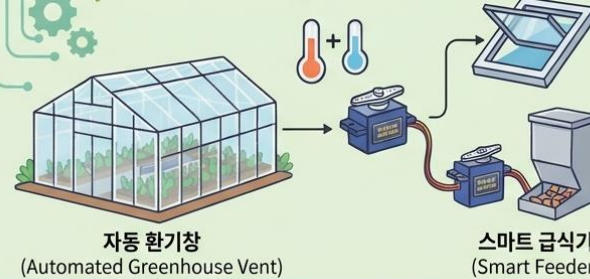
Robotics & Articulated Joints (로봇공학 & 관절 제어)



다축 로봇 팔
(Multi-axis Robotic Arm)

로봇 관절 제어
(Robotic Joint Control)

Smart Farm (스마트 팜)



자동 환기창
(Automated Greenhouse Vent)

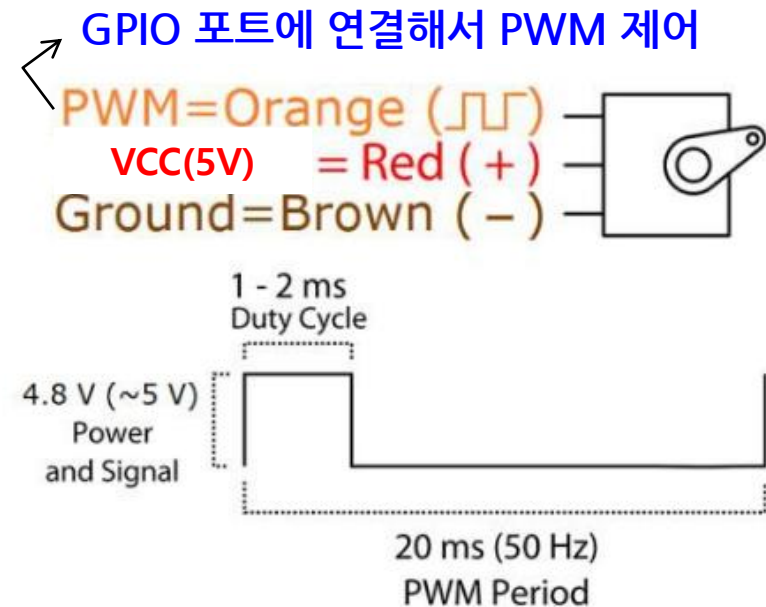
스마트 급식기
(Smart Feeder)

1. 서보모터의 구성

□ 서보모터(Servo Motor)

- 명령에 따라 정확한 위치와 속도를 제어할 수 있는 모터
- 응용: 자동화 생산 시스템, 로봇, 가전제품, CCTV 카메라, 캠코더, 프린터등 사용
- 종류: 표준 서보모터 (회전각도 $0^{\circ} \sim 180^{\circ}$)

- 서보모터에는 3개의 단자가 나와 있음



[서보모터 규격]

2. PWM을 이용한 서보모터 기본 제어

□ 실습1) 서보모터 제어 - 회로 연결하기

- 서보모터의 GPIO는 18번 사용

3V3	(1)	(2)	5V
GPIO2	(3)	(4)	5V
GPIO3	(5)	(6)	GND
GPIO4	(7)	(8)	GPIO14
GND	(9)	(10)	GPIO15
GPIO17	(11)	(12)	GPIO18
GPIO27	(13)	(14)	GND

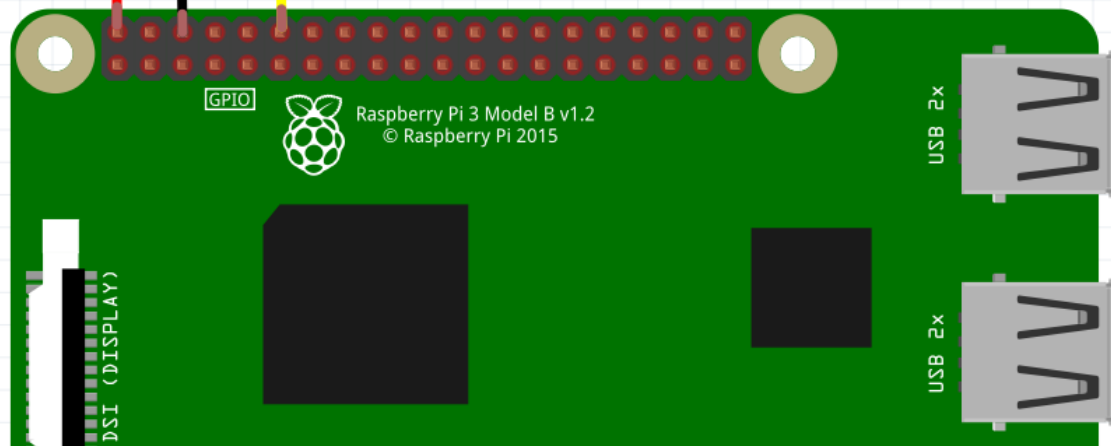
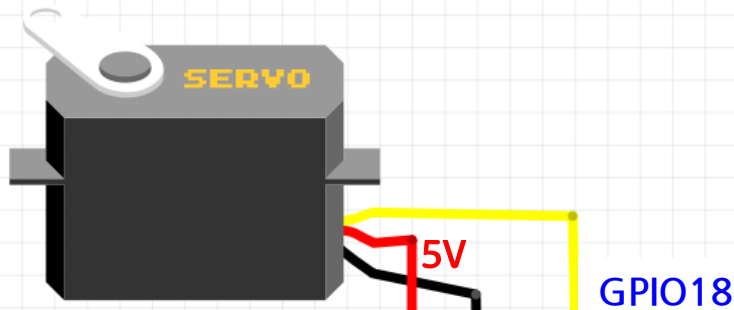
(GPIO18)

PWM=Orange (┐┘)

5V Vcc = Red (+)

Ground=Brown (-)

(GND)

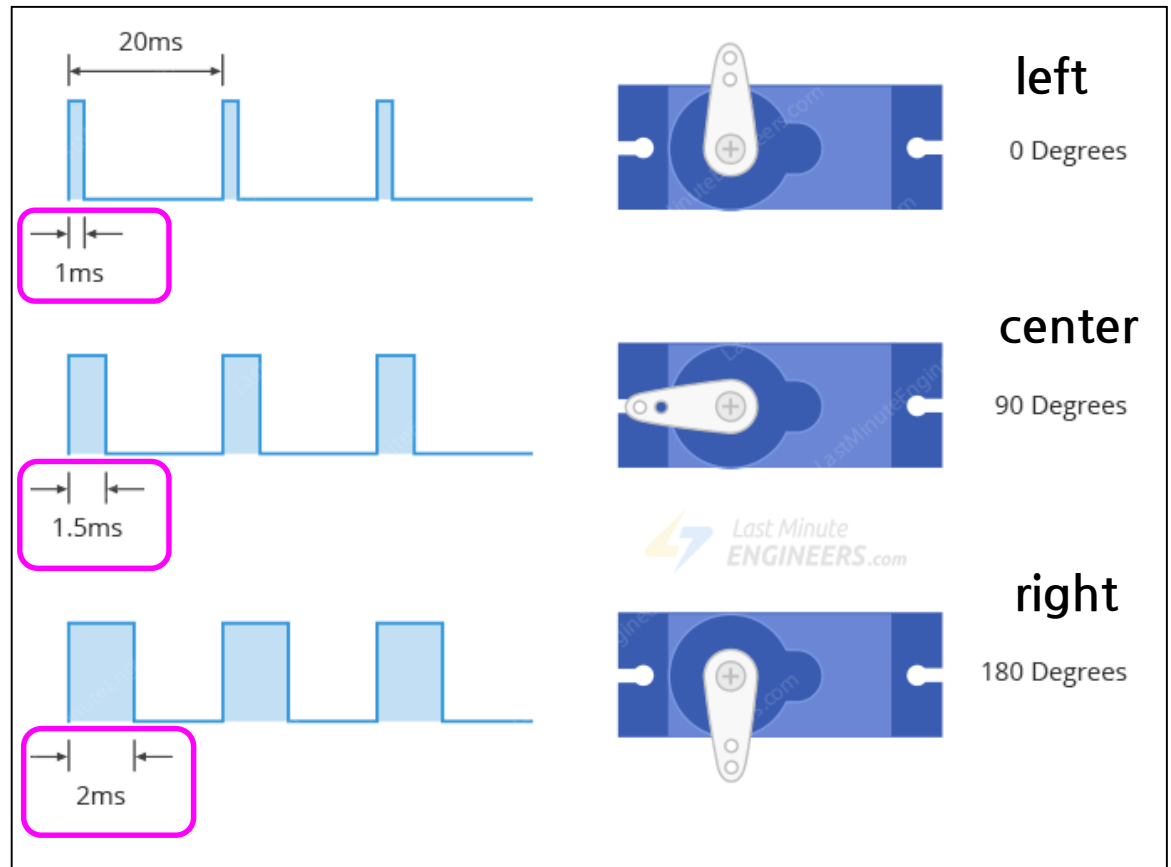
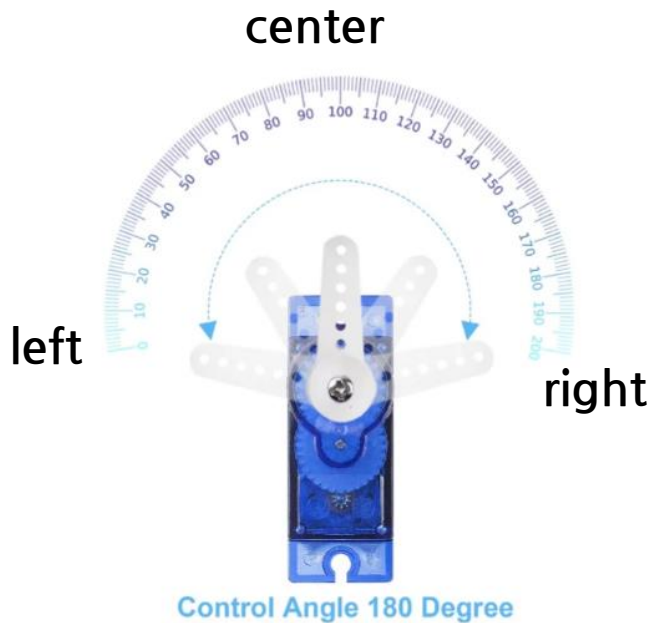


2. PWM을 이용한 서보모터 기본 제어

□ 서보모터의 회전 각도는 아래 그림과 같이 PWM (펄스폭변조) 방식으로 제어

- 서보모터(Servo Motor)의 규격을 확인해 보시다

[서보모터 규격]



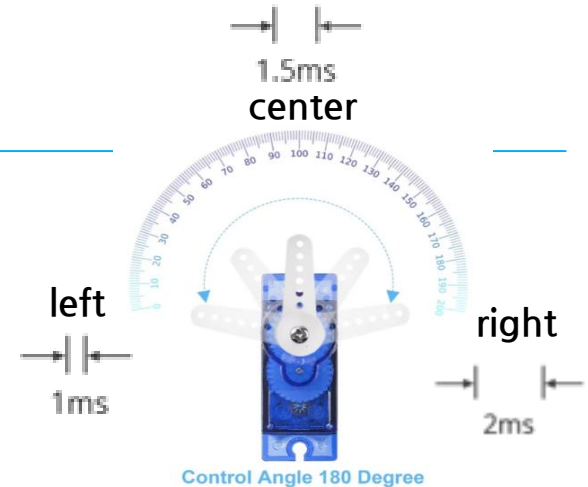
2. PWM을 이용한 서보모터 기본 제어

□ gpiozero 라이브러리를 이용한 서보모터 제어 방법

```
# 서보모터 제어하기
from gpiozero import Servo

# SG90 서보모터 객체 생성 (GPIO18번)
servo = Servo(18)
```

※ Servo default value
 servo = Servo(18, min_pulse_width = 1.0/1000, max_pulse_width=2.0/1000)



```
def min(self):
    self.value = -1.0
```

servo.min()은
 servo.value = -1.0
 과 동일하게
 작동하는 코드

	방식1) 방향만 제어	방식2) 정확한 각도를 제어
Left(0도)	servo.min() # left(값은 -1.0)	servo.value = -1.0
Center(90도)	servo.mid() # center(값은 0.0)	servo.value = 0.0
Right(180도)	servo.max() # right(값은 1.0)	servo.value = 1.0

※ gpiozero는 각도(0~180) 대신 -1.0에서 1.0 사이의 소수점 값으로 서보모터를 제어

2. PWM을 이용한 서보모터 기본 제어

□ 실습1) 서보모터의 방향을 제어해 봅시다 (규격값 그대로 사용하기)

파일명: servo.py

```
# 서보모터 제어하기
from gpiozero import Servo
import time

# SG90 서보모터 객체 생성(GPIO18번)
servo = Servo(18)

print("서보모터 위치제어 실습(전선을 정면으로 보기)")
print('='*45)
print("l:left(0도), c:center(90도), r:right(180도)")
print("프로그램 종료는 'q'입력")
print('='*45)
```

0도 ~180도까지 정확히 동작하나요?

2가지 문제를 해결해 봅시다.

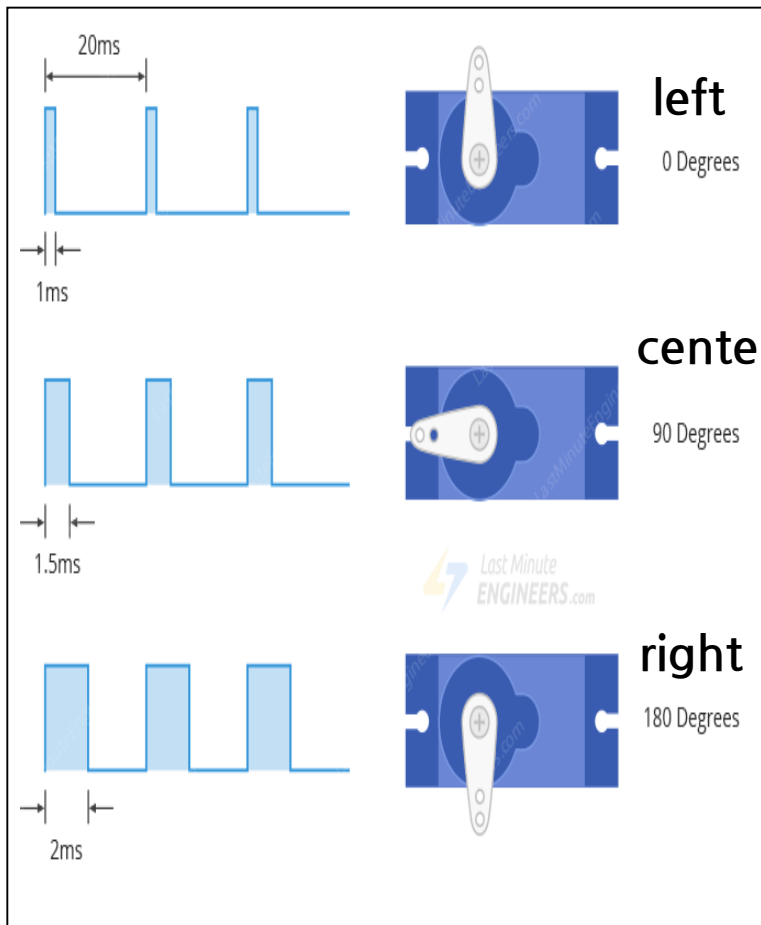
- 정확한 위치제어
- 떨림 방지

```
try:
    while True:
        # 사용자로 위치 입력받기
        command = input("원하는 위치를 선택하세요(l/c/r/q)")
        if command == 'l':
            print("Left 위치로 이동 (value: -1.0)")
            servo.min()
            #servo.value = -1.0      # 동일하게 작동
        elif command == 'c':
            print("Center 위치로 이동 (value: 0.0)")
            servo.mid()
            #servo.value = 0.0      # 동일하게 작동
        elif command == 'r':
            print("Right 위치로 이동 (value: 1.0)")
            servo.max()
            #servo.value = 1.0      # 동일하게 작동
        elif command == 'q':
            print("프로그램을 종료합니다")
            break
        else:
            print("잘못된 입력입니다")
            # 모터가 해당위치까지 물리적으로 회전할 최소한의 시간 주기
            time.sleep(0.5)
except KeyboardInterrupt:
    pass
finally:
    servo.close()
```

2. PWM을 이용한 서보모터 기본 제어

❑ 실습1) 서보모터의 보정한 Duty Cycle(듀티비) 수치입니다

[서보모터 규격]



보정결과

➡ 0.5ms (min_pulse_width=0.5/1000)

➡ 1.5

➡ 2.5 (max_pulse_width=2.5/1000)

2. PWM을 이용한 서보모터 기본 제어

□ 실습1) 보정값을 적용해 서보모터의 방향을 제어해 봅시다

* servo.py 코드 보완

- 기존 코드에서 이 부분을 수정해 줍시다. 서보모터의 min ~ max 범위 재 설정

* default 는 min_pulse_width = 1.0/1000, max_pulse_width=2.0/1000



```
from gpiozero import Servo
import time

# SG90 객체 생성 (GPIO 18번)
servo = Servo(18, min_pulse_width=0.5/1000, max_pulse_width=2.5/1000)

print("서보모터 위치제어 실습(전선이 뒤로 가게 놓고 정면 보기)")
print("=" * 45)
print(" l: left(0도), c: center(90도), r: right(180도)")
print(" 프로그램 종료는 'q' 입력")
print("=" * 45)
```

- 코드를 실행해서 0도 ~180도 사이를 정확히 동작하는지 확인해 주세요.

3. 서보모터 떨림 해결

□ 실습2) 서보모터의 떨림(jitter)을 해결해 봅시다

서보모터가 떨리는 가장 큰 이유는 목표 각도에 이미 도달했는데도 라즈베리파이가 백그라운드(softPWM)에서 펄스 신호를 계속해서 밀어 넣고 있기 때문

→ gpiozero 라이브러리에서는 모터 이동이 끝난 후 펄스 신호를 잠시 끊어주는 `detach()` 메서드를 제공, 모터는 떨림(jitter)을 멈추고 Idle 상태가 됨.

```
elif command == 'q':  
    print("프로그램을 종료합니다.")  
    break  
else:  
    print("잘못된 입력입니다")  
  
# 모터가 해당 위치까지 물리적으로 회전할 최소한의 시간주기  
time.sleep(0.5)  
servo.detach() # 이동후 펄스신호를 끊어 떨림방지  
  
except KeyboardInterrupt:  
    pass  
finally:  
    servo.close()  
    print("프로그램을 종료합니다.")
```

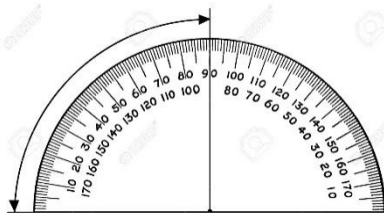
모터를 원하는 방향으로 이동 후 `detach()`를 이용해 PWM 펄스 신호를 끊어 떨림(jitter)을 방지하기

4. 서보모터의 정확한 각도 제어

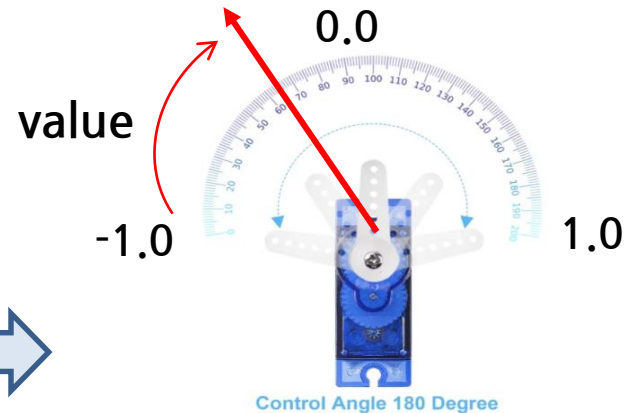
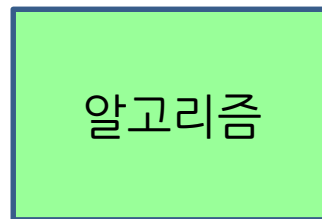
□ 실습3) 사용자로부터 각도를 입력받아 서보모터 위치 제어하기

이 문제를 해결할 여러분의 알고리즘은?

angle: $0^{\circ} \sim 180^{\circ}$



angle $\rightarrow$ value

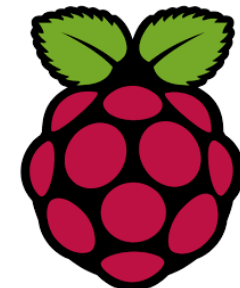


PWM



45도로 이동해 줘!

value값은?



4. 서보모터의 정확한 각도 제어

□ 실습3) 사용자로부터 각도를 입력받아 서보모터 위치 제어하기

* 파일명: servo_angle.py

angle → value



서보모터의 정확한 각도 조절하기

```
from gpiozero import Servo
import time
```

SG90 서보모터 객체 생성(GPI018번), 보정값 적용

```
servo = Servo(18,
               min_pulse_width=0.5/1000,
               max_pulse_width=2.5/1000)
```

```
print("서보모터 각도제어 프로그램(value 방식)")
print('='*30)
print("프로그램 종료는 'q'입력")
print('='*30)
```

```
try:
    while True:
        user_input = input("각도를 입력하세요(0~180):")

        if user_input == 'q':
            print("프로그램을 종료합니다")
            break

        angle = float(user_input)

        # 0~180도 사이인지 검사하기
        if 0 <= angle <= 180:
            # 0~180도 각도를 입력받아 -1.0~1.0사이의 value로 변환하는 변환식
            # 예시: 0도 -> -1.0, 90도 -> 0.0, 180도 -> 1.0
            servo_value = # 핵심 변환 로직을 구현해 보세요!
            servo.value에 값 대입하기 # 계산된 값 value 속성에 대입

            print(f"입력각도:{angle} -> value:{servo_value:.2f}")
            print("서보모터 이동하기")
            time.sleep(0.5) # 모터 회전시간 제공
            servo.detach() # 떨림 방지
        else:
            print("0~180사이 각도를 입력하세요")
except KeyboardInterrupt:
    pass
finally:
    servo.close()
```

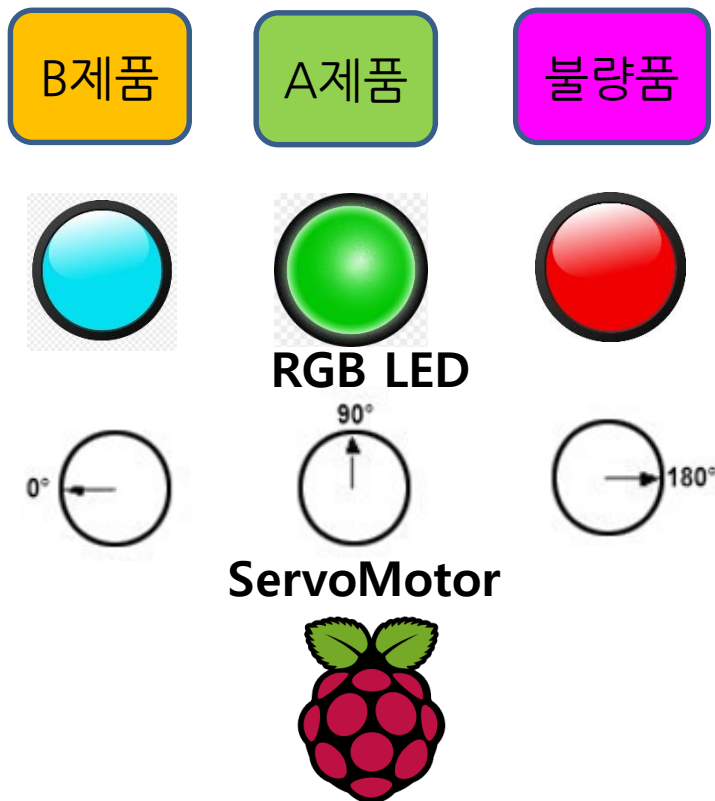
5. 응용실습과제

□ 스마트 팩토리 자동 분류 시스템 구현하기 : 서보모터와 RGBLED 응용하기

[조건] 물류 컨베이어 시스템에서 상품이 지나갑니다.

센서가 'A제품', 'B제품', '불량품'으로 분류하면, 미리 약속된(딕셔너리 DB는 제공) 방식으로 RGB LED와 서보의 방향을 제어해 보세요

단, 센서는 사용자 입력으로 대신합니다.



스마트팩토리 자동분류 시스템 시작

=====

1: A제품 통과, 2: B제품 통과, 3: 불량품 감지
0: 시스템 정지(초기화), q: 시스템 빠져나오기

=====

센서코드 입력: 1

센서감지: A제품(중앙라인)

상품 전송완료

센서코드 입력: 2

센서감지: B제품(왼쪽라인)

상품 전송완료

센서코드 입력: 3

센서감지: 불량품(오른쪽라인)

상품 전송완료

5. 응용실습과제

□ 스마트 팩토리 자동 분류 시스템 구현하기

* 파일명: 이름_4주차_응용실습과제.py

```
# 4주차 응용실습과제: 스마트팩토리 자동분류 시스템
from gpiozero import RGBLED, Servo
import time

# [빈칸] SG90 구동 펄스폭 지정 및 객체 생성(GPI018)
servo = _____ 펄스폭 보정값 적용하기
# [빈칸] RGBLED GPIO pin 지정 및 객체 생성
rgb = _____ GPIO핀은 red=17, green=22, blue=27

# 스마트팩토리 DB 딕셔너리로 저장
factory_db = {
    '1': ((0,1,0), 0, "A제품(중앙라인)", # Green, 서보위치: 0(mid)
    '2': ((0,0,1), -1, "B제품(왼쪽라인)", # Blue, 서보위치: -1(min)
    '3': ((1,0,0), 1, "불량품(오른쪽라인)", # Red, 서보위치: 1.0(max)
    '0':((0,0,0), 0, "시스템 초기화(시스템 정지)" # Off, 서보위치: 0(mid)
}

print("스마트팩토리 자동분류 시스템 시작")
print('='*40)
print("1:A제품 통과, 2:B제품 통과, 3:불량품 감지 ")
print("0: 시스템 초기화(시스템 정지), q: 시스템 빠져나오기")
print('='*40)
```

5. 응용실습과제

□ 스마트 팩토리 자동 분류 시스템 구현하기

```
try:
    # 시스템 초기상태 지정
    # [빈칸] DB에서 RGB 초기상태 가져와서 색상 지정하기
    rgb.color = key로 해당 value 가져오기
    # [빈칸] DB에서 서보모터 초기상태 가져와서 위치 지정하기
    servo.value = key로 해당 value 가져오기
    time.sleep(0.5) # 모터 이동시간 보장
    # [빈칸] 서보모터의 떨림(jitter) 방지
    #

while True:
    # 사용자로부터 센서 입력값 받기
    item = input("센서코드 입력:")
    if item == 'q':
        print("시스템 가동을 중지합니다")
        break
```

```
if item in factory_db:
    # [빈칸] 코드(키)에 해당하는 정보(값) DB에서 가져와 변수에 저장하기
    color_val, servo_pos, status = DB에서 키로 가져오기
    print(f"센서감지: {status}")

    # [빈칸] RGBLED 색상 구동하기
    color 이용

    # [빈칸] 서보모터 위치 구동하기
    value 이용
    time.sleep(0.5) # 모터 이동시간 보장
    # [빈칸] 서보모터의 떨림(jitter) 방지
    #

    print("상품 전송완료")
else:
    print("잘못된 코드입니다")
except KeyboardInterrupt:
    print("\n 사용자 강제 종료 감지")
finally:
    # 사용한 리소스 완전히 반납
    servo.close()
    rgb.close()
    print("스마트 팩토리 자동 분류시스템 종료하기")
```

4주차 과제 제출 안내

□ 4주차 실습과제

- 과제: 3교시 응용실습 과제를 제출해 주세요
- 제출방법: CTL 사이트 4주차 과제 제출란에 제출해 주세요
- 제출파일: 이름_4주차_응용실습과제.py
- * 실습 시 잘 안된 부분이 있다면 *comment*를 남겨 주세요



수고 많았습니다!